

Dynamic Arrays: How Python Lists Grow

Programming and Data Structures — Week 2, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain, step by step, what happens internally the moment `list.append` exceeds the array's current capacity; contrast a naive "grow by one slot" resizing strategy against a "grow by doubling" strategy using worked numeric totals, not just intuition; distinguish clearly between an array's length — the number of elements actually stored — and its capacity — the number of slots currently reserved in memory; and predict, by hand, the capacity of a dynamic array after any given sequence of appends.

Outline

Today covers a moving-house analogy for what a resize actually is; the puzzle left open at the end of Lecture 1 — how a fixed block of memory grows at all; a hand simulation of exactly what happens on each append call; a comparison of a naive grow-by-one strategy against Python's actual grow-by-doubling approach, backed by real arithmetic; the distinction between an array's length and its capacity, confirmed by measurement; a complete worked trace of 16 consecutive appends; an in-class quiz; and a summary.

The moving-house analogy

Picture owning a 1-bedroom flat that has just become too small because a friend has moved in. You cannot bolt on an extra bedroom — Lecture 1 established that the space next door is not yours to take. The only option is to rent an entirely new, larger flat, move every single box across, and give up the old flat. Because moving itself is exhausting and costly, it makes sense to move to a place with considerable spare room — not just one bedroom more than you currently need — so that you do not have to repeat the whole exhausting process again immediately. This is precisely the reasoning behind a dynamic array doubling its capacity rather than growing by one slot at a time, which the rest of this lecture makes mathematically precise.

The puzzle

```
scores = []  
for i in range(1_000_000):  
    scores.append(i)
```

Lecture 1 established that a fixed array cannot safely grow in place. Yet this code plainly works, appending a million times without the programmer ever declaring a size in advance. Something is happening on each `append` call that reconciles these two facts, and that mechanism is this lecture's entire subject.

Simulating it by hand: step 1

```
capacity = 1, length = 0  
arr = [ _ ]
```

```
scores.append(10)
```

```
capacity = 1, length = 1  
arr = [ 10 ]
```

Start with a tiny array of capacity 1. The first `append(10)` fits in the single reserved slot: length becomes 1, capacity stays 1, nothing special happens yet.

Simulating it by hand: step 2, the resize

```
scores.append(20)    # capacity is full (1/1) - must resize
```

Appending 20 finds capacity already full — 1 slot used out of 1. Four things happen, in this order: a brand new array is allocated with double the capacity (2 slots); every existing element is copied from the old array to the new one (just 10 here); the new element (20) is inserted; and the old 1-slot array is discarded. The result:

```
capacity = 2, length = 2  
arr = [ 10, 20 ]
```

Simulating it by hand: step 3

```
scores.append(30)    # capacity is full (2/2) - must resize
```

Before: capacity=2, arr=[10, 20]
Allocate: new array, capacity=4
Copy: [10, 20, _, _]
Insert: [10, 20, 30, _]
After: capacity=4, length=3

The third append repeats the same four-step process: capacity 2 is already full, so a new array of capacity 4 is allocated, both existing elements (10 and 20) are copied across, and 30 is inserted into the third slot. One slot (index 3) now sits reserved but unused — this is exactly the “spare room” the moving-house analogy described.

Simulating it by hand: steps 4 and 5

```
1 scores.append(40)    # fits (3/4, one free slot) → no resize  
2 scores.append(50)    # full (4/4) → resize to capacity 8, copy 10,20,30,40, insert 50
```

Tracing five appends shows resizing happens only when the current capacity is exhausted — most calls just write into an already-reserved slot. The capacity sequence produced is 1, 2, 4, 8: each resize doubles the previous capacity rather than growing by a fixed amount.

Counting the actual work done

Tabulating the exact work done on each of the five appends traced so far makes the cost pattern concrete: append 1 does no copying at all; append 2 copies 1 existing element; append 3 copies 2; append 4 copies 0 (it fits); append 5 copies 4. Total copying across these five appends: $0 + 1 + 2 + 0 + 4 = 7$ element-copies, for 5 appends — an average of 1.4 copies per append, already suggesting the total stays proportional to the number of appends, not to its square.

Why doubling, and not “add one slot each time”

A “grow by exactly one slot” strategy resizes on *every single append* after the first, copying the entire existing array each time. Over n appends, that is roughly $1 + 2 + 3 + \dots + n \approx n^2/2$ total copy operations — $O(n^2)$ total work for n appends, or $O(n)$ work *per append* on average, which is disastrous for a million-element loop. Grow-by-doubling resizes only $\log_2 n$ times total, and the total copying across all resizes is bounded by $n + n/2 + n/4 + \dots < 2n$ — $O(n)$ total work for n appends. This difference — n^2 versus n total work — is the entire reason every practical dynamic array implementation, including Python’s `list`, doubles rather than increments.

Putting real numbers on the difference

Substituting a realistic n of one million into both formulas makes the difference impossible to ignore. Grow-by-1 gives roughly $n^2/2 = 500,000,000,000$ total copy operations — half a trillion. Grow-by-doubling gives fewer than $2n = 2,000,000$ — two million. The grow-by-1 strategy would make a one-million-element loop that should finish in well under a second instead take hours, purely from unnecessary copying — this is not a theoretical concern, it is the difference between a program that works and one that appears to hang.

Length vs. capacity: two different numbers

```
import sys
scores = []
for i in range(6):
    scores.append(i)
    print(len(scores), sys.getsizeof(scores))
```

`len(scores)` reports the length — the number of elements actually stored, which increases by exactly one on every `append`, without exception. `sys.getsizeof(scores)` reflects the underlying capacity, which only jumps upward on the calls that trigger a resize and stays flat otherwise. Running this loop and watching both numbers side by side is the clearest way to see that length and capacity are two genuinely different quantities, not two names for the same thing.

Reading the actual output

```
len=1  size=... (capacity allocated for 1)
len=2  size=... (capacity jumped - resize happened)
len=3  size=... (capacity jumped again)
len=4  size=... (SAME as len=3's size - free slot used, no resize)
```

```
len=5  size=... (capacity jumped again)
len=6  size=... (SAME as len=5's size - free slot used)
```

Running the loop from the previous slide produces a size column that jumps upward only at certain lengths and stays flat in between — exactly the doubling pattern traced by hand earlier in this lecture, now confirmed by actually measuring Python’s real behavior rather than only reasoning about it abstractly. This is a useful habit generally: whenever a claimed complexity result is testable directly, measure it.

The picture: three snapshots of the same list

1. Capacity 2, full — append(30) arrives



resize →

2. New block, capacity 4 — copy, then insert



Copied: 10, 20 (from old block) — Inserted: 30 (new element) — Discarded: old 2-slot block

3. Result: capacity 4, length 3, one free slot remains



Next append(40) fits here — no resize needed

Resizing always allocates a brand new block, copies every existing element, then inserts — never grows the old block in place.

Each resize allocates a new block, copies every existing element into it, and only then adds the new one — the old block is discarded afterward.

MTech addition — capacity growth factor choices

CPython’s actual growth factor is not a clean $2\times$ — for large lists it approaches roughly $1.125\times$ to reduce wasted memory, using a formula documented in CPython’s own `listobject.c`. The amortized $O(n)$ result proved formally next lecture holds for *any* fixed growth factor greater than 1, not specifically 2 — the constant only affects how much memory sits unused between resizes, trading memory efficiency against resize frequency.

MTech addition — the general growth-factor formula

$$\text{total copies over } n \text{ appends} \leq n \cdot \frac{g}{g-1}, \quad g > 1$$

For a general growth factor $g > 1$ (so capacity multiplies by g on each resize), the total copying across n appends is bounded by $n \cdot \frac{g}{g-1}$. Setting $g = 2$ gives the familiar bound $2n$ used earlier in this lecture. As $g \rightarrow 1$ (barely growing at all), this bound blows up toward infinity, recovering the $O(n^2)$ behavior of grow-by-1 in the limit — confirming that the choice of growth factor is really a continuous dial between “waste less memory” and “resize less often,” not a binary choice.

Full worked trace — 16 appends from empty

Tracing capacity across a full sequence of 16 appends from an empty array confirms the pattern completely. Resizes occur only at appends 1, 2, 3, 5, and 9 — five resizes total across sixteen appends, with capacity following the doubling sequence 1, 2, 4, 8, 16. Every append between resizes (4, 6 through 8, and 10 through 16) is a plain $O(1)$ write into an already-reserved slot. This table is worth reproducing by hand for a different starting capacity as practice before the lab.

Exercise

Starting from capacity 1 and doubling on each resize, trace the capacity after each of 10 appends. At which specific append numbers (1st, 2nd, 3rd, ...) does a resize actually occur, and how many total resizes happen across all 10 appends? As a further exercise, extend the full worked trace table from this lecture out to the 20th append.

Quiz — read the trace

Given the observed capacity sequence 1, 1, 2, 2, 4, 4, 4, 4, 8, 8, 8, ... (one entry per append), work out after how many total appends exactly 3 resizes have occurred, before checking the answer.

Quiz — answer

Reading the capacity sequence: capacity goes from 1 to 2 at the 2nd append (resize 1), from 2 to 4 at the 3rd append (resize 2), and from 4 to 8 at the 5th append (resize 3). So exactly 3 resizes have occurred by the 5th append — matching the doubling pattern where resizes happen at appends 2, 3, 5, 9, 17, and so on.

Summary

A dynamic array resizes by allocating a new, larger block, copying every existing element into it, and only then inserting the new element — the old block is discarded. Doubling the capacity on each resize keeps total copying across n appends at $O(n)$, rather than the $O(n^2)$ a naive grow-by-one strategy would produce — at one million appends this is the difference between roughly two million copy operations and roughly half a trillion. `len()` reports elements stored; capacity is slots currently reserved — these are genuinely different numbers, and confusing them is a common source of confusion when reading Python's memory behavior. Next lecture proves the $O(n)$ claim formally and introduces the term “amortized $O(1)$ ” for what a single append costs on average.